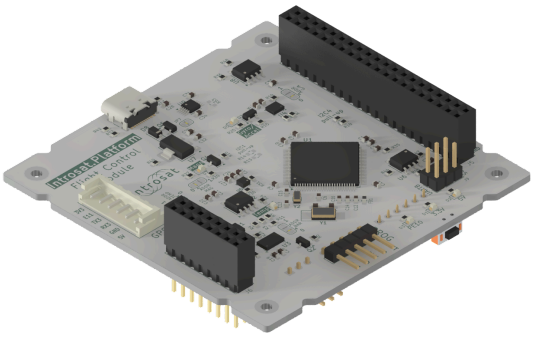
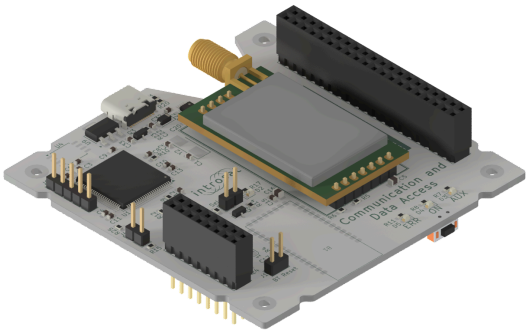
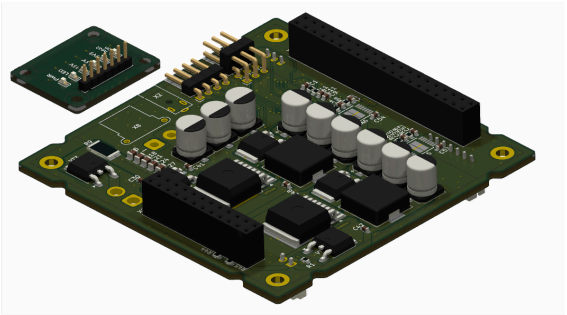
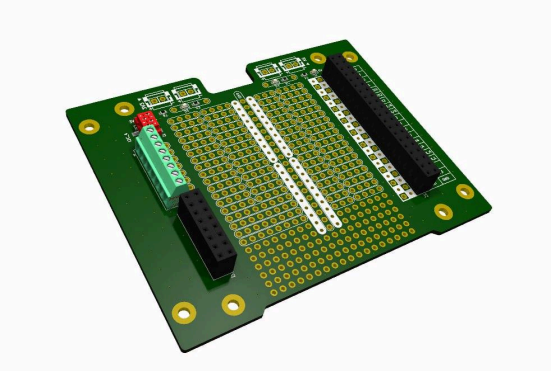
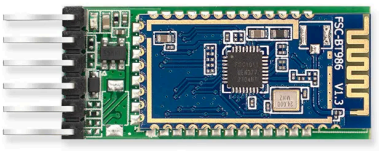
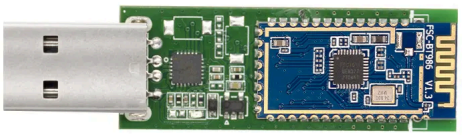
IntroSat.Platform: a very short guide

Service system modules

Flight Controller Module	
Communication module	
Power supply module	

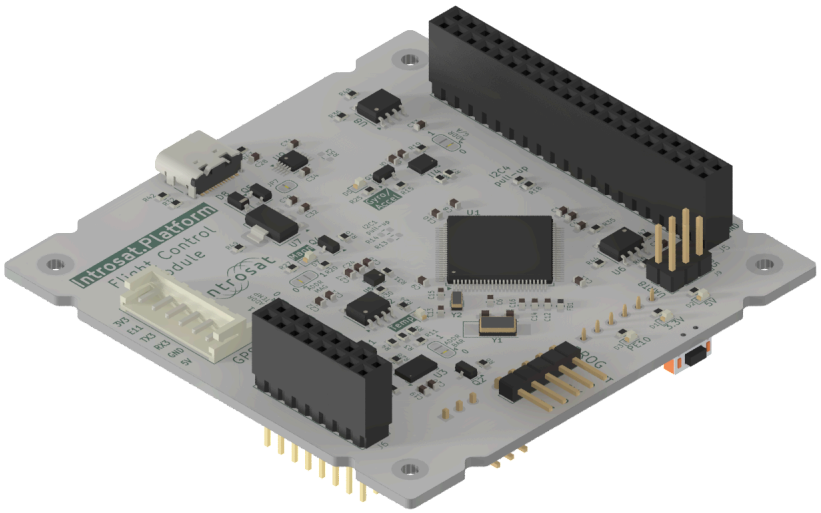
Interface module	

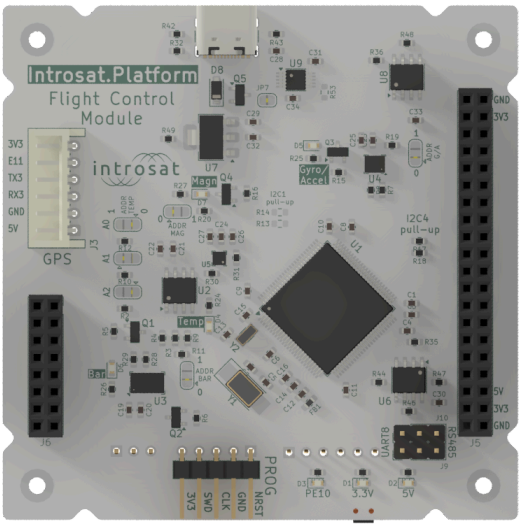
Auxiliary devices

ST-Link programmer	
USB-UART converter	
Bluetooth module	
Bluetooth module with built-in USB-UART converter	

Flight Controller Module

General information





Appearance of the Flight Controller Module Board
In addition to the controller, this board houses the main sensors required for obtaining telemetry during flight:

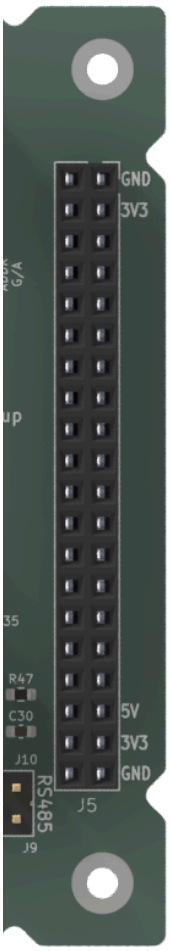
- LSM6DS3 performs two functions:
- Accelerometer: Measures the device's acceleration.
- Gyroscope: Measures the device's angular velocity.
- LIS3MDL: A magnetometer that measures magnetic field induction.
- MS5611: A barometer that measures pressure and calculates approximate altitude.
- LM75A: A temperature sensor that monitors the board's temperature.

IntroBus

To connect service module boards in a stack, a proprietary bus called IntroBus is used. It consists of two double-row through-hole pin headers: IntroBus_S (16-pin) and IntroBus_L (40-pin).

> The IntroBus_S connector is for internal use only (designated as J6 on the flight controller module board). Connection to this connector is strictly prohibited.

The IntroBus_L connector (J5) is used to connect payload devices to the microcontroller.



IntroBus_L connector (J6) on the flight controller module

IntroBus_L_2x20 (FCM)			
GND	GND	GND	**GND**
+3V3_IS	3V3	3V3	+3V3_IS
OPAMP_OUT	PE7	PB0	ADC
OPAMP_INM	PE8	PB1	ADC
OPAMP_INP	PE9	PB15	MOSI2
ADC / DFSDM_CKIN	PC0	PB14	MISO2
ADC / DFSDM_DATIN	PC1	PB10	SCK2
ADC / DFSDM_CKOUT	PC2_C	PE0/A	RX8/A
ADC	PC3_C	PE1/B	TX8/B
COMP_INM	PC4	PD9	RX3
COMP_OUT	PC5	PD8	TX3
HRTIM_CHA1	PC6	PB12	RX5
HRTIM_CHA2	PC7	PB13	TX5
COMP_INP	PB2	PD13	SDA4
DAC_EXT	PB9	PD12	SCL4
DAC_OUT1	PA4	CAN_H	CAN_H
DAC_OUT2	PA5	CAN_L	CAN_L
+5V_IS	5V	5V	+5V_IS
+3V3_IS	3V3	3V3	+3V3_IS
GND	GND	GND	**GND**
View of the board from above			

Connecting sensors:

All service sensors located on the flight controller board are connected via the I2C1 bus. It is a service unit and serves only for communication within the service systems unit; it is not possible to connect a payload to it.

The I2C1 bus uses the following microcontroller pins:

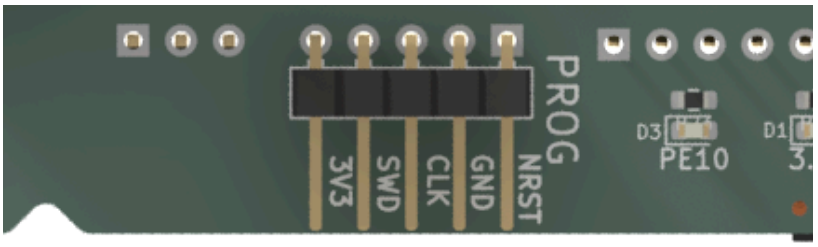
SDA1	PB7
SCL1	PB6

Also, several pins of the microcontroller are connected to some sensors to receive additional signals from them.

Sensor output	Microcontroller pin	Appointment
LM75A: вывод OS	PE2	Interruption
LSM6DS3: вывод INT1	PA15	Interruption
LIS3MDL: вывод INT	PB3	Interruption
LIS3MDL: вывод DRDY	PB4	Data availability

Connector for firmware and debugging

The main way to download the firmware to the microcontroller is the corner connector PLS-5R. It is signed as PROG on the board.



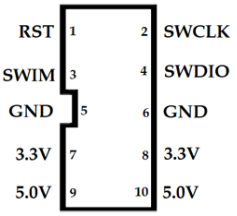
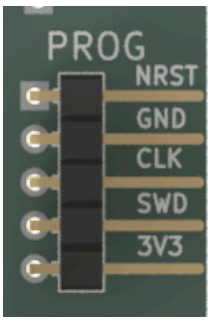
Microcontroller firmware connector

To flash the microcontroller, an ST-Link programmer must be connected to this connector using 4 "mom-mom" wires.



ST-Link Programmer

The firmware is loaded via the SWD (Serial Wire Debug) interface. Also, if the development environment has such functionality, this interface is used for debugging – working with the microcontroller in line-by-line program execution mode with the ability to view the current values of variables and the contents of registers. Connection is carried out according to the table:

ST-Link	FCM
 	
SWCLK >--- ---> CLK SWDIO >--- ---> SWD GND >--- ---> GND	

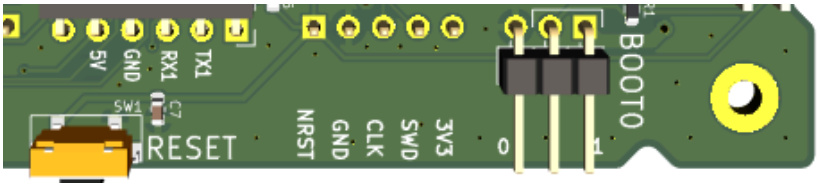
If no other power source is connected to the flight controller board, you can connect the 3.3V pins of the ST-Link programmer to the 3V3 pin on the PROG connector of the board. Thus, the service systems of the board (and the microcontroller) will be powered by 3.3 V.

If necessary, you can use the fifth pin to reset the microcontroller (pin RST on the programmer and NRST on the FCM board).

BOOT0 Jumper

There is a BOOT0 jumper on the board, which can be used to physically put the microcontroller into bootloader mode: in this mode, the microcontroller expects the firmware to be loaded into it via one of the available interfaces. More information about the STM32 loader can be found in the Application Note AN2606:

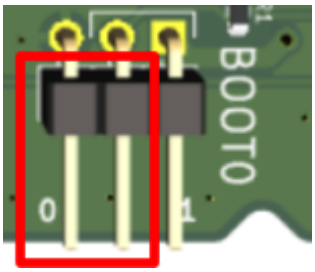
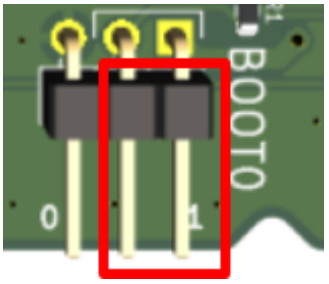
https://www.st.com/resource/en/application_note/an2606-introduction-to-system-memory-boot-mode-on-stm32-mcus-stmicroelectronics.pdf



BOOT0 jumper on the back of the flight controller board

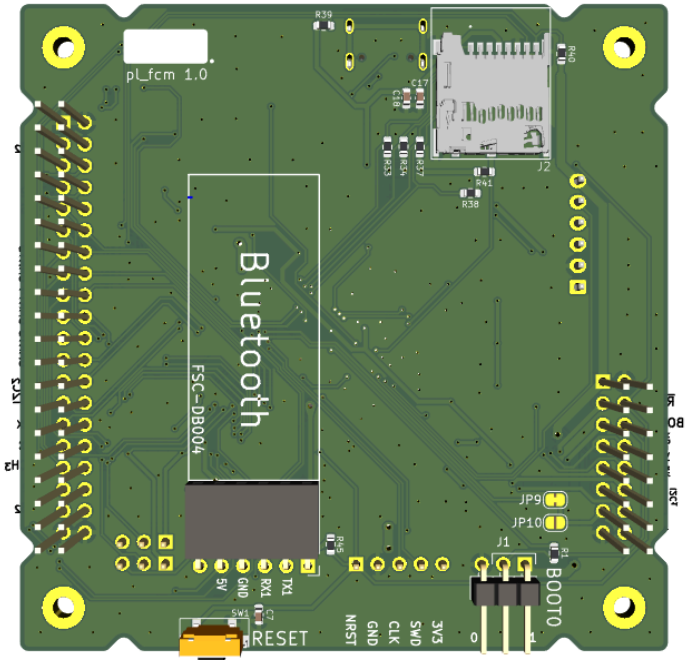
Before starting work with the flight controller board, it is necessary to set the **BOOT0** jumper to position **0** using a jumper (that is, combine the contact with silkscreen printing 0 and the middle contact).

To force the microcontroller into bootloader mode, it is necessary to move the jumper to position 1. If the jumper is switched on when the power is on, then press the RESET button. After loading the firmware in the desired way, it is necessary to return the jumper to position 0 so that the microcontroller can execute the code loaded on it.

Normal operation of the microcontroller	Microcontroller in bootloader mode
	

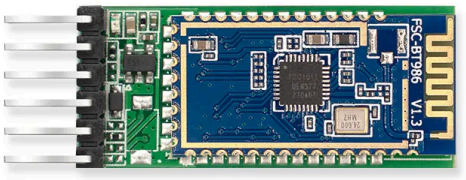
Information exchange via UART (main communication channel)

Bluetooth communication



The reverse side of the flight controller board

On the back of the flight controller board there is a connector for connecting the Bluetooth module FSC-DB004-BT986. The module is inserted into the connector in only one way, but before connecting, it is recommended to make sure that the silkscreen printing of the module contacts corresponds to the silkscreen printing next to the connector on the flight controller board (while RX1 on the board should go to the TX module and vice versa, TX1 on the board to the RX module).

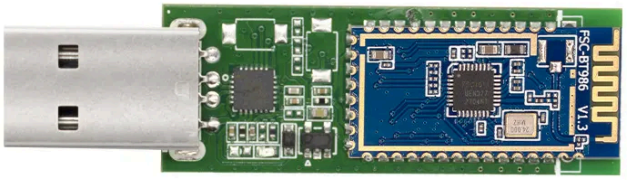


Bluetooth-модуль FSC-DB004-BT986

The microcontroller's UART1 is output to the Bluetooth connector, specifically the following pins:

- RX1 → PA10
- TX1 → PA9

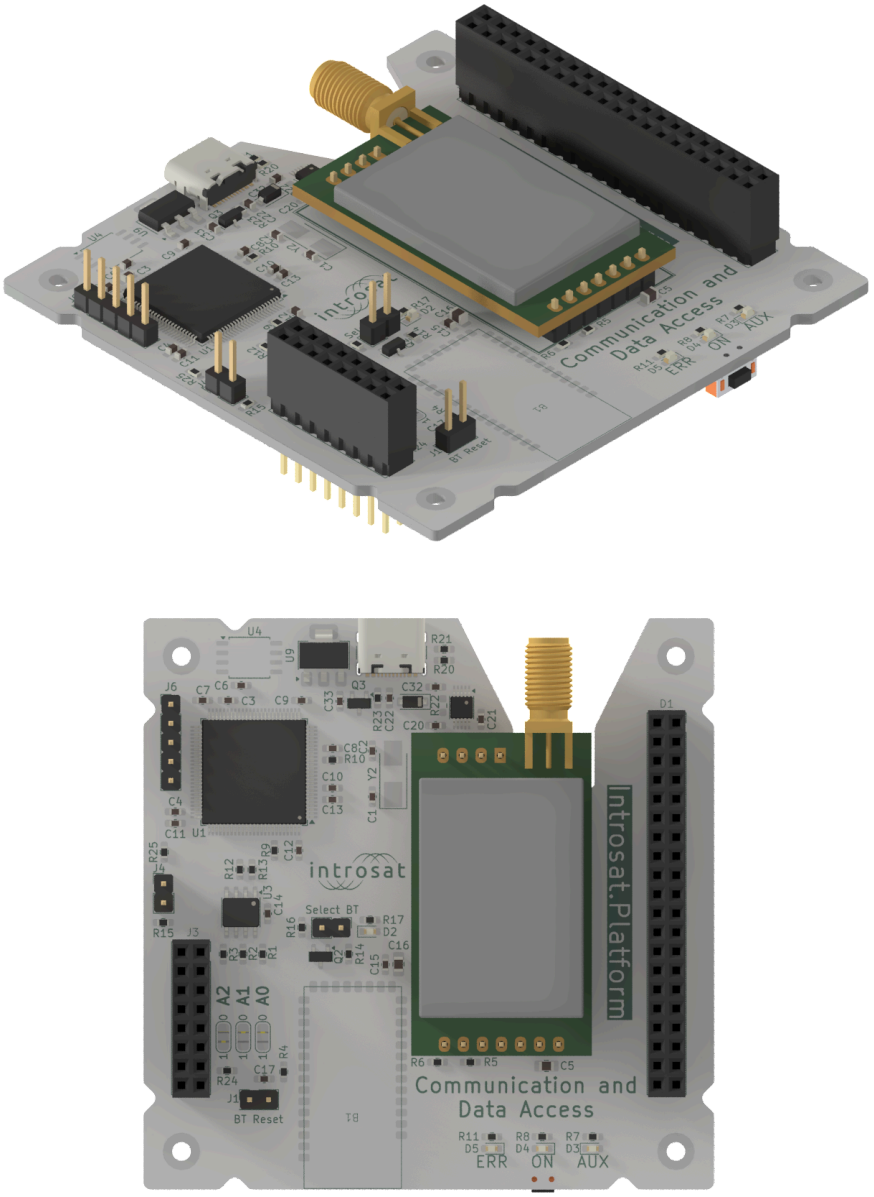
As a response, a Bluetooth module with a built-in USB-UART converter FSC-DB005-BT986 is connected to the PC.



FSC-DB005-BT986

Communication and Data Access Module

General information



The appearance of the communication board

The communication card allows you to exchange data with a ground station using a radio module operating using the LoRa protocol.

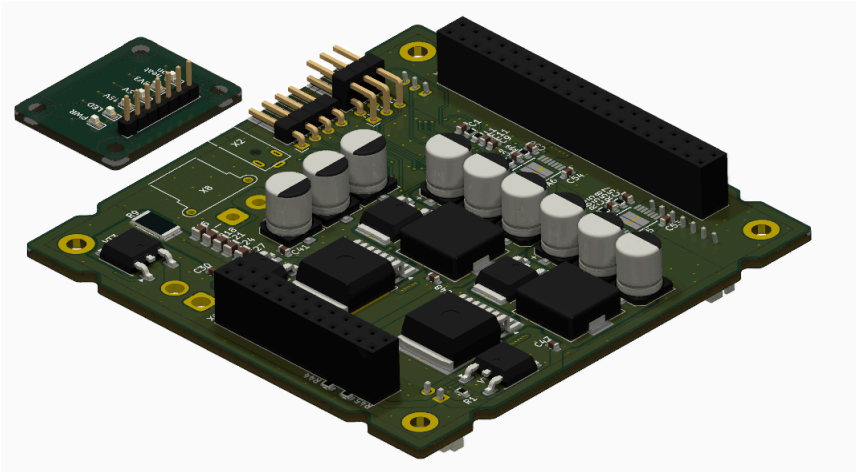
Microcontroller model	STM32H750VBT6
Radio module	Ebyte E32-433T30D

In addition to the microcontroller and the radio module, the board also features an LM75AD temperature sensor, with its I2C address set to 0x4C. The I2C bus connected to this sensor corresponds to the I2C1 bus on the flight controller board.

The radio module on the board operates in the 410–441 MHz frequency range, with a configurable step of 1 MHz. This allows for 32 independent channels, preventing interference between them. Channel selection should take into account the noise levels of specific frequencies at the launch site to ensure reliable communication.

Transmitting data over a radio channel without an antenna connected can damage the radio module. It is recommended to always connect the antenna.

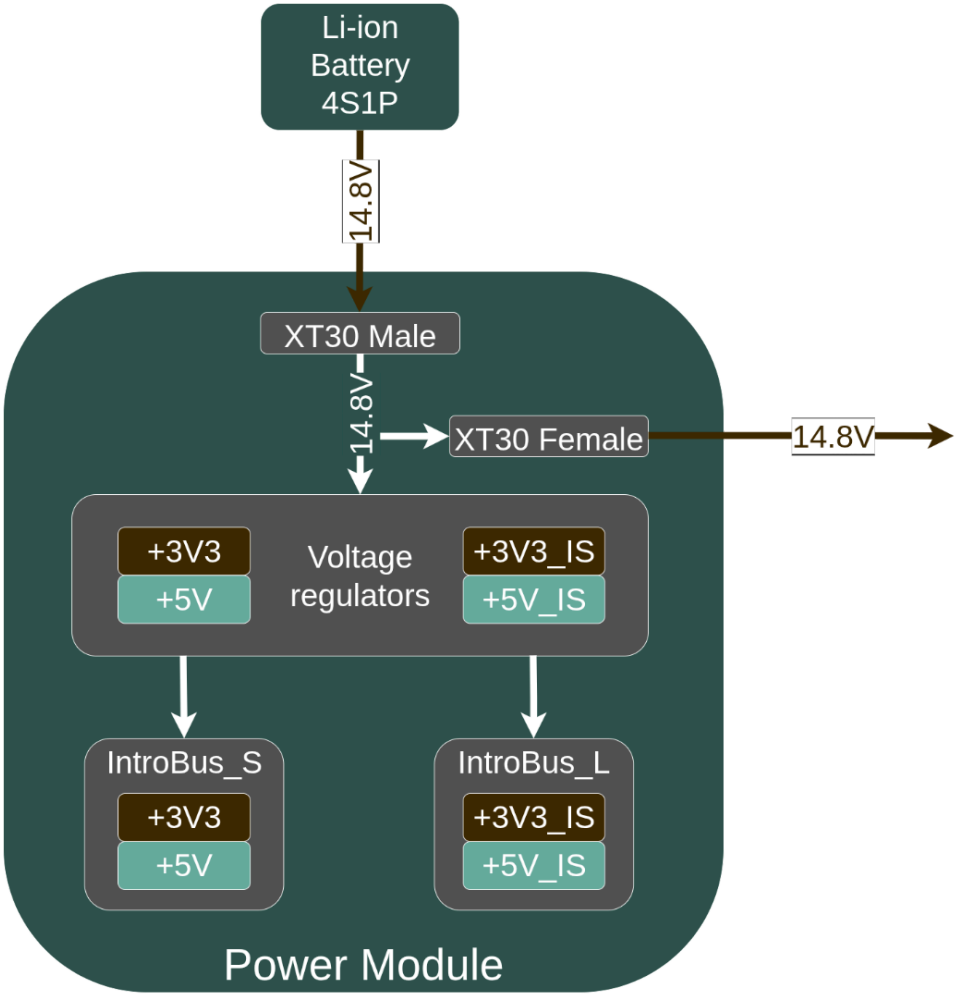
Power Module



Appearance of the power supply board (separate display module)

The power supply module provides power to the IntroSat.Platform constructor. The power supply module board is always at the top of the stack. The power source can be an assembly of 4S1P Li-ion batteries (4 batteries connected in series) with a rated voltage of 14.8 V, or a 12 V power supply connected to an electrical network.

Attention!
The assembly of heated batteries is given at the Orbita tournament. A 12 V power supply is used during preparation.



Simplified functional diagram of the power supply module

The input voltage is supplied to the XT30-Male connector (14.8 V from the battery assembly or 12 V from the power supply) and is further divided into several channels:

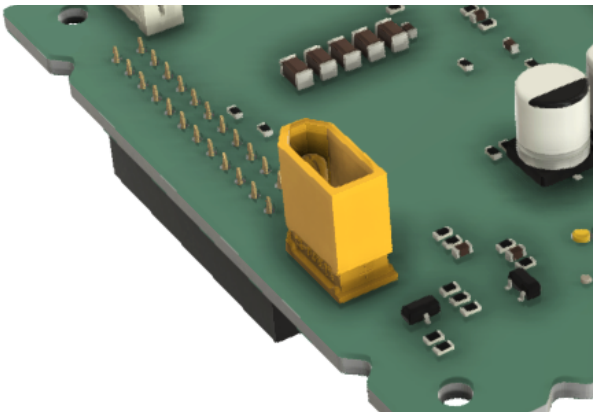
Characteristics of the power channels

Attention! Do not short-circuit or exceed the specified currents to avoid failure of the devices.

Power channel	**Max current, A**
Unstable voltage 14.8 V	6
Stable voltage 5 V	3

Stable voltage 3.3 V	3

Connecting the power supply



Connector for connecting the power supply

The XT30-Male connector, silk-screened as X1, is used to connect the power supply.

The 12 V power supply must be connected via an adapter to the XT30-Female.



Power supply 12 V



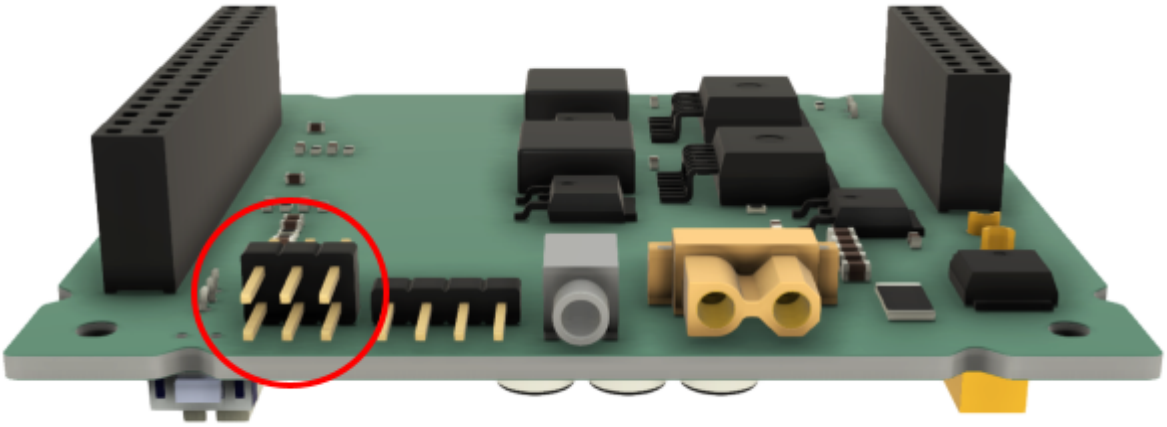
Adapter for XT30 power supply

Attention!

Do not use 3.3V and 5V power lines on the St-Link programmer, USB-UART converter, or other external devices connected to the constructor when the power supply module is connected to the stack.

Switching on power channels

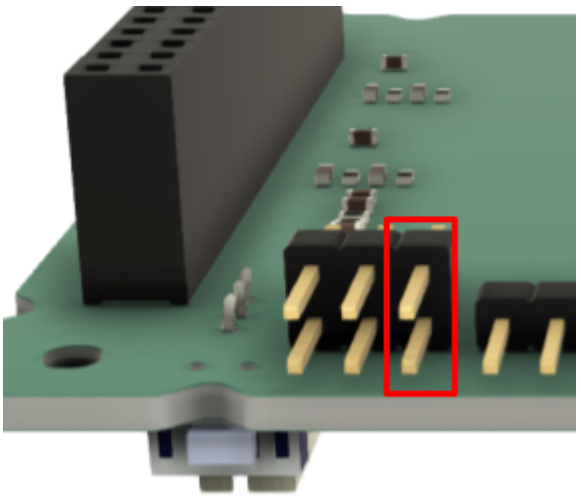
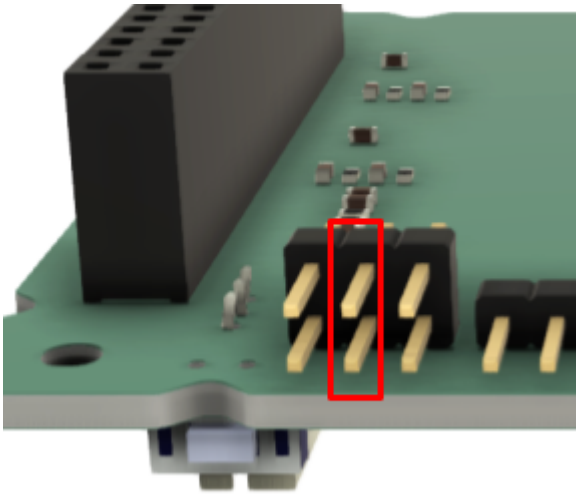
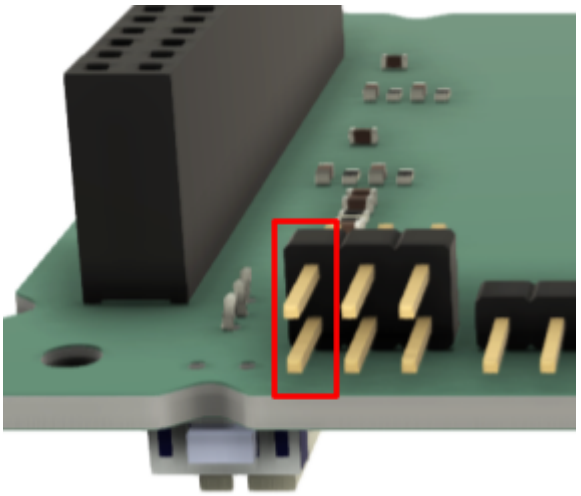
The power channels for the payload are switched on by installing jumpers on the six-pin X6 connector. The jumpers are installed perpendicular to the board.



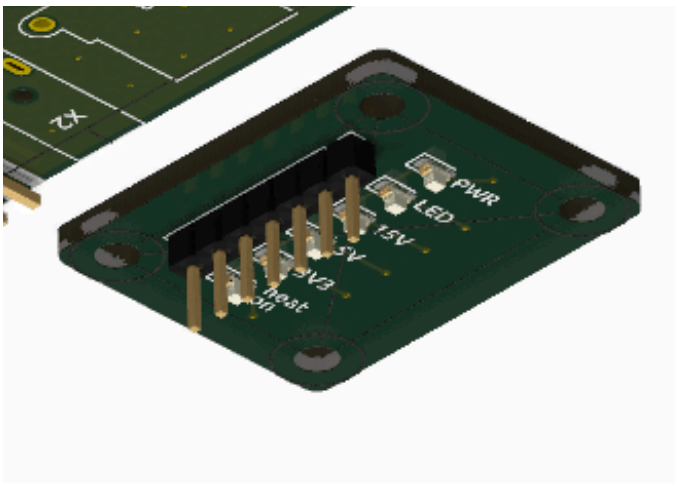
The X6 connector responsible for switching on channels

The table below shows the correspondence between the power supply channel and the contacts that need to be closed to turn it on.:

Channel	Location of the jumper

3.3 B (+3V3_IS)	
5 B (+5V_IS)	
Unstable 14.8 B	

When the channel is switched on, the corresponding LED on the display module connected to the power supply board lights up.

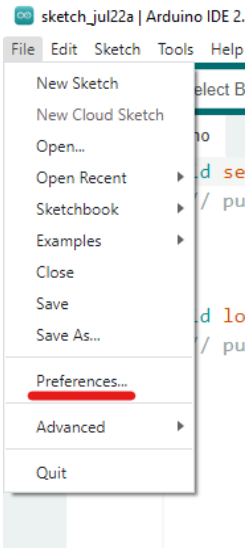


The display module

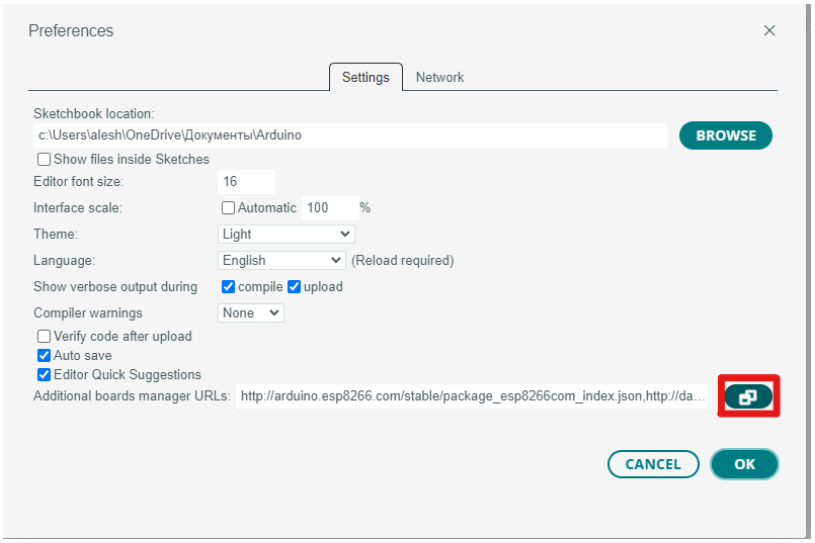
Installing software and libraries

To write the program code and firmware of the flight controller, you can use the Arduino IDE software (download link). To work with the STM32 family of cards, you must also install an additional card support package. To do this, you need to insert an additional link for the card manager in the Arduino IDE settings.

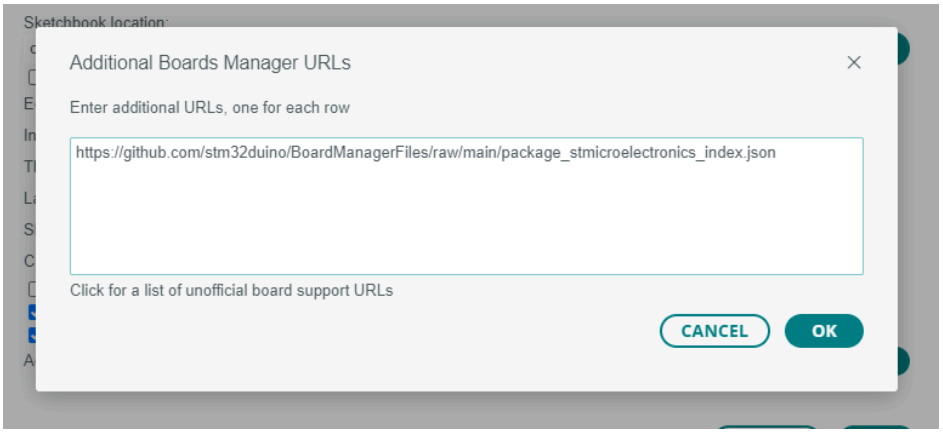
https://github.com/stm32duino/BoardManagerFiles/raw/main/package_stmicroelectronics_index.json



Settings Menu

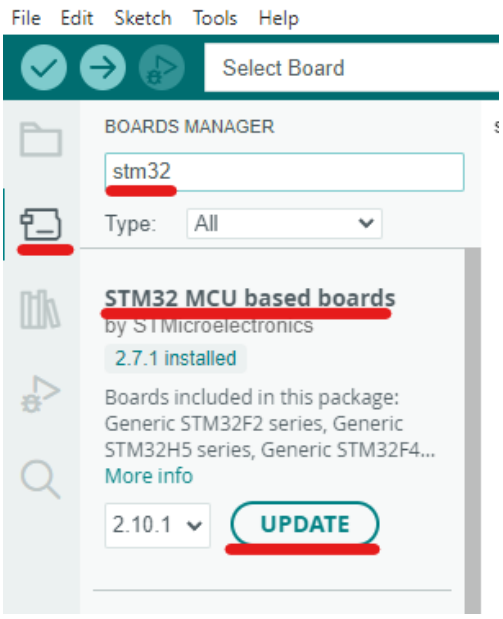


Opening the additional links window for the card manager



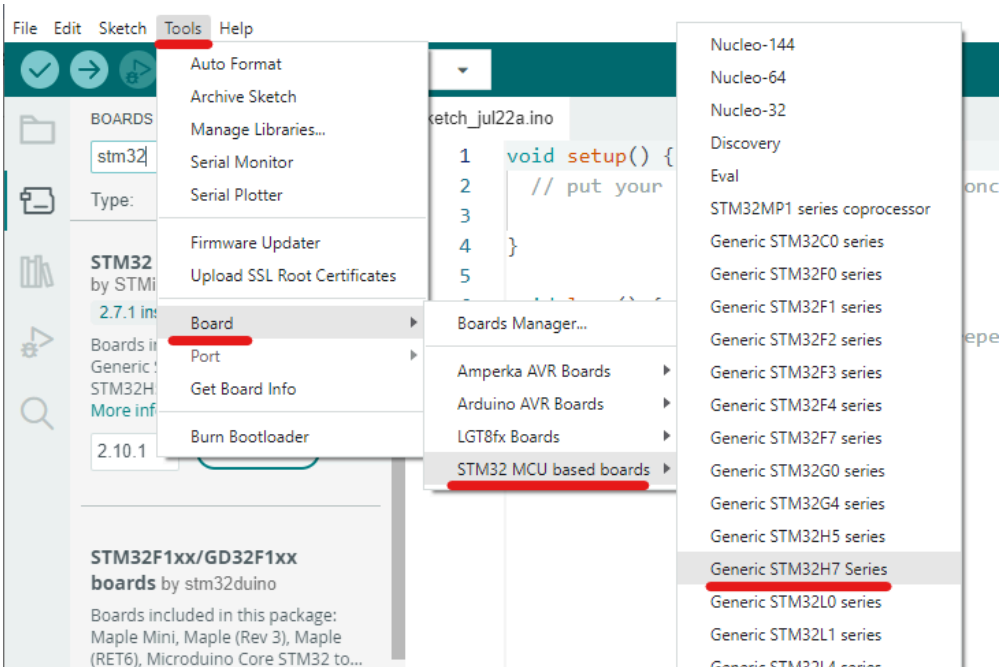
Inserted link

After that, you need to open the Board Manager, type “stm32” in the search and install the “STM32 MCU based boards" package. The installation may take some time depending on the internet connection.



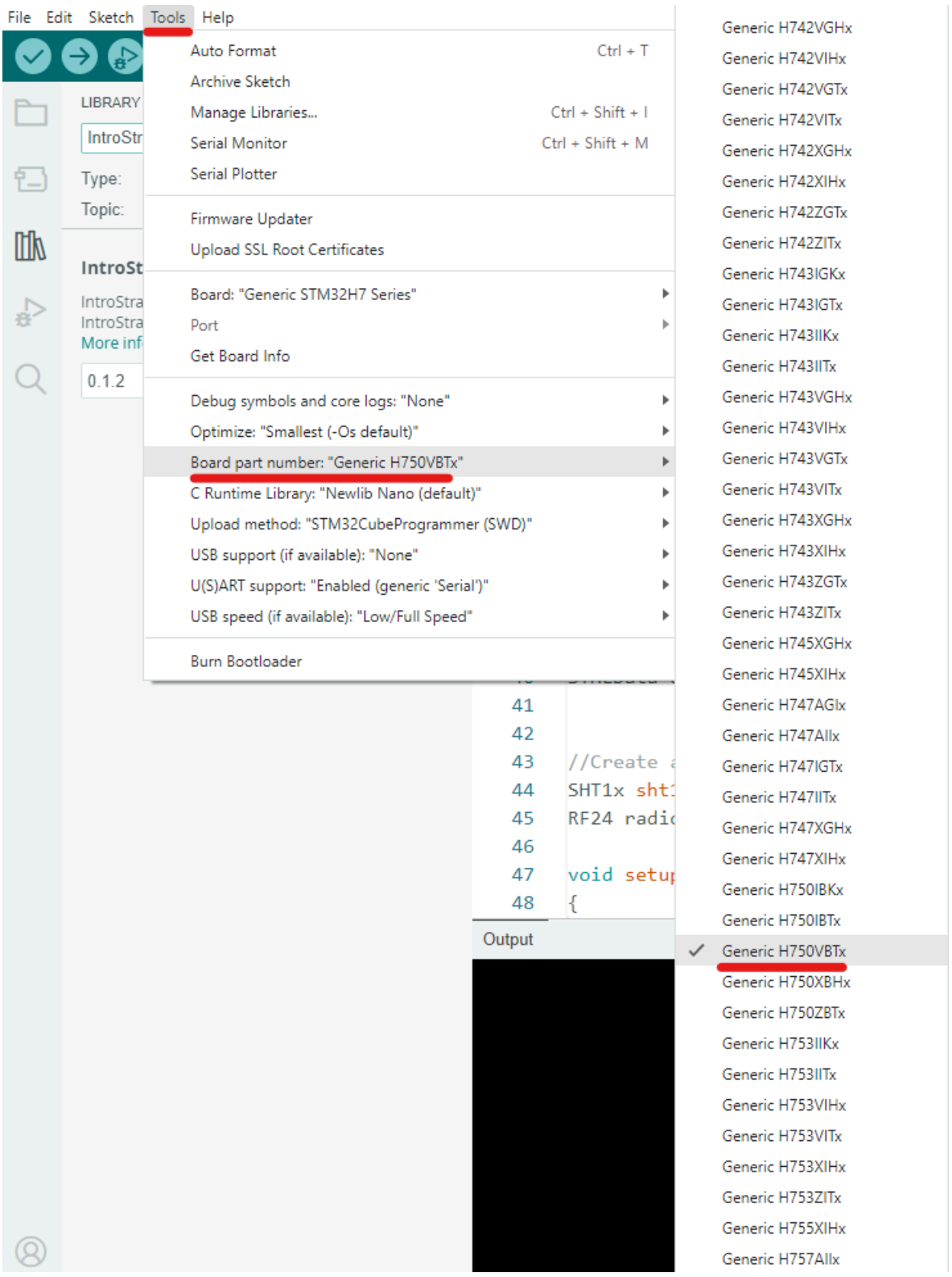
Installing the board package

You can now select the appropriate series of microcontrollers in the Tools menu.



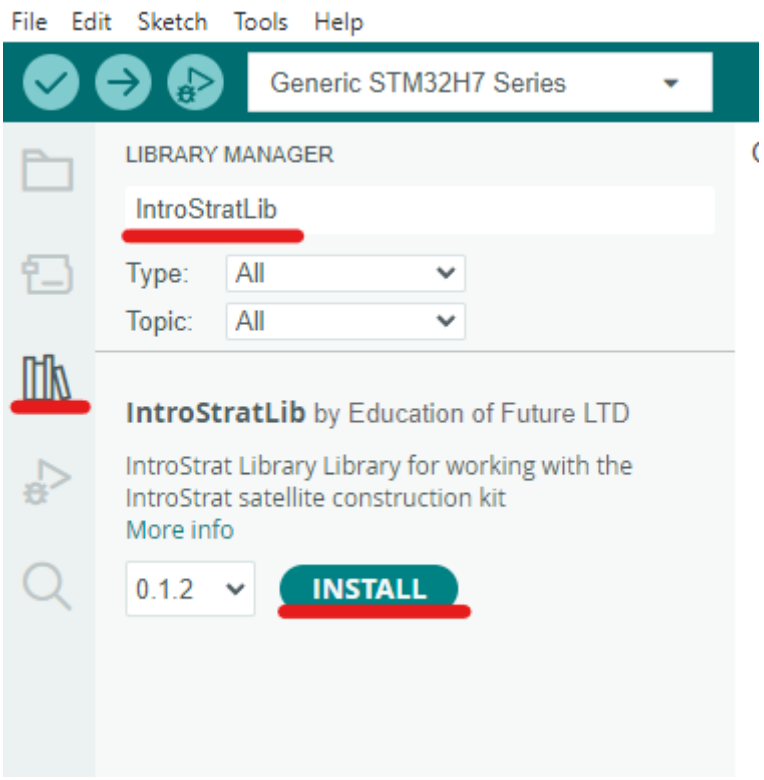
Choosing a family of microcontrollers

Next, in the same menu, select the installed controller (if the necessary microcontroller is not in the list, scroll through it below):



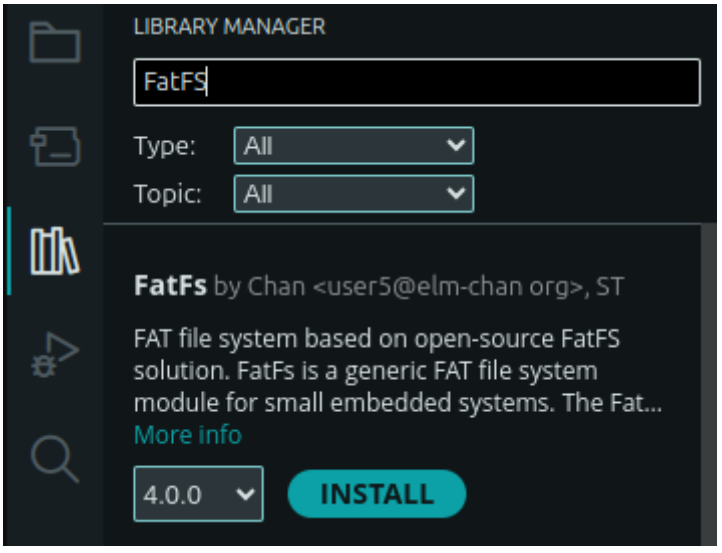
Choosing a microcontroller

In the future, it is recommended to use the IntroStratLib library for convenient work with the platform. It can be installed through the library manager in the Arduino IDE.



Installing the IntroStratLib library

It is also recommended to install two libraries to save logs and useful data to the SD card: FatFs and STM32SD. They can also be found in the library manager.



Installing the FatFs Library



Installing the STM32SD library

After completing the configuration of the Arduino IDE, you also need to download and install the STM32CubeProgrammer. With it, it will be possible to flash STM32 microcontrollers via the Arduino IDE. Download link (requires free registration on the website):

https://www.st.com/en/development-tools/stm32cubeprog.html

If the link above doesn't work, choose the right OS and download the software from the Google Drive:

https://drive.google.com/drive/folders/16j7ZNJIxbnB-YA9xYJRHTKJxJhtnI8BM?usp=sharing

It may also require installing Java in addition, you need to do it as well.

Getting started with UART

To work with any of these interfaces, you must create an object of the `HardwareSerial` class with an indication of the pins on which the interface is being created. In general, the creation of an object looks like this:

```
HardwareSerial MySerial(RX_pin, TX_pin);
```

The following objects may be required for the interfaces used on the flight controller board:

```
HardwareSerial Serial1(PA10, PA9);
HardwareSerial Serial3(PD9, PD8);
HardwareSerial Serial5(PB12, PB13);
```

```
HardwareSerial Serial8(PE0, PE1);
```

After that, you can work with each of them in the same way as with the usual Serial object in Arduino. You can use any other name instead of `Serial1`.

Initialization

In the `setup()` function UART settings must be set:

```
Serial1.begin(115200, SERIAL_8E1);
```

This line initializes the previously set **UART1** at **115200** baud, using **8** data bits, **Even** parity check, and **one** stop bit.

Data transmission

Standard Serial methods can be used to transfer data.

```
Serial1.print("Hello world!"); // passing a string
Serial1.write(0x14); // byte transfer
```

Receiving data

The STM32duino implements buffering of incoming data, which allows you to use the standard `available()` method to check for incoming data.

```
if (Serial1.available() > 0) {
    uint8_t data = Serial1.read(); // we read 1 byte from the
incoming data
    // doing something with data
}
```

I2C

I2C (**I**nter-**I**ntegrated **C**ircuit) - a common protocol for connecting peripheral devices to a common bus. It uses two lines:

SDA - for receiving and transmitting data

SCL - for clocking purposes

There should always be *one master* device on the bus, and up to 127 *slaves*. Any data exchange is *initialized by the master device*, whether it is requesting data from a sensor or writing a value to a peripheral device. In this case, the

microcontroller on the FCM board acts as the master device. Two I2C interfaces are derived from it:

I2C1 - the system bus. All sensors and devices installed on the platform are connected to it.

I2C4 - a bus for connecting payload devices.

Getting started with I2C

To work with any of these interfaces, you need to create a `TwoWire` class object with an indication of the pins on which the interface is being created. In general, it looks like this:

```
TwoWire MyWire(PB7, PB6);
```

The following objects will be required for the system bus and the payload connection bus:

```
TwoWire Wire1(PB7, PB6);
TwoWire Wire4(PD13, PD12);
```

After that, you can work with each of them in the same way as with the usual Wire object in Arduino. You can use any other name instead of `Wire1`.

Initialization

In the `setup()` function I2C must be initialized:

```
Wire1.begin();
```

Calling the `begin()` method without parameters initializes the interface in master mode and allows you to work with peripheral devices.

Data exchange

To communicate with peripheral devices (for example, sensors), it is recommended to use objects and methods from the IntroStratLib library.

Four service sensors are installed on the FCM board:

Sensor name	Measured values	The library used	I2C bus address
LSM6DS3 (gyroscope and accelerometer)	angular velocity (deg/s), linear acceleration (g)	`LSM6DS3.h`	0x6A

LIS3MDL (magnetometer)	magnetic induction (Gauss)	`LIS3MDL.h`	0x1C
MS5611 (barometer)	pressure (mbar), temperature (°C)	`MS5611.h`	0x77
LM75A (temperature sensor)	Temperature (°C)	`LM75A.h`	0x4A

IntroStratLib description

All service sensors located on the flight controller board are connected via the I2C1 bus. You can read more about working on the I2C interface above.

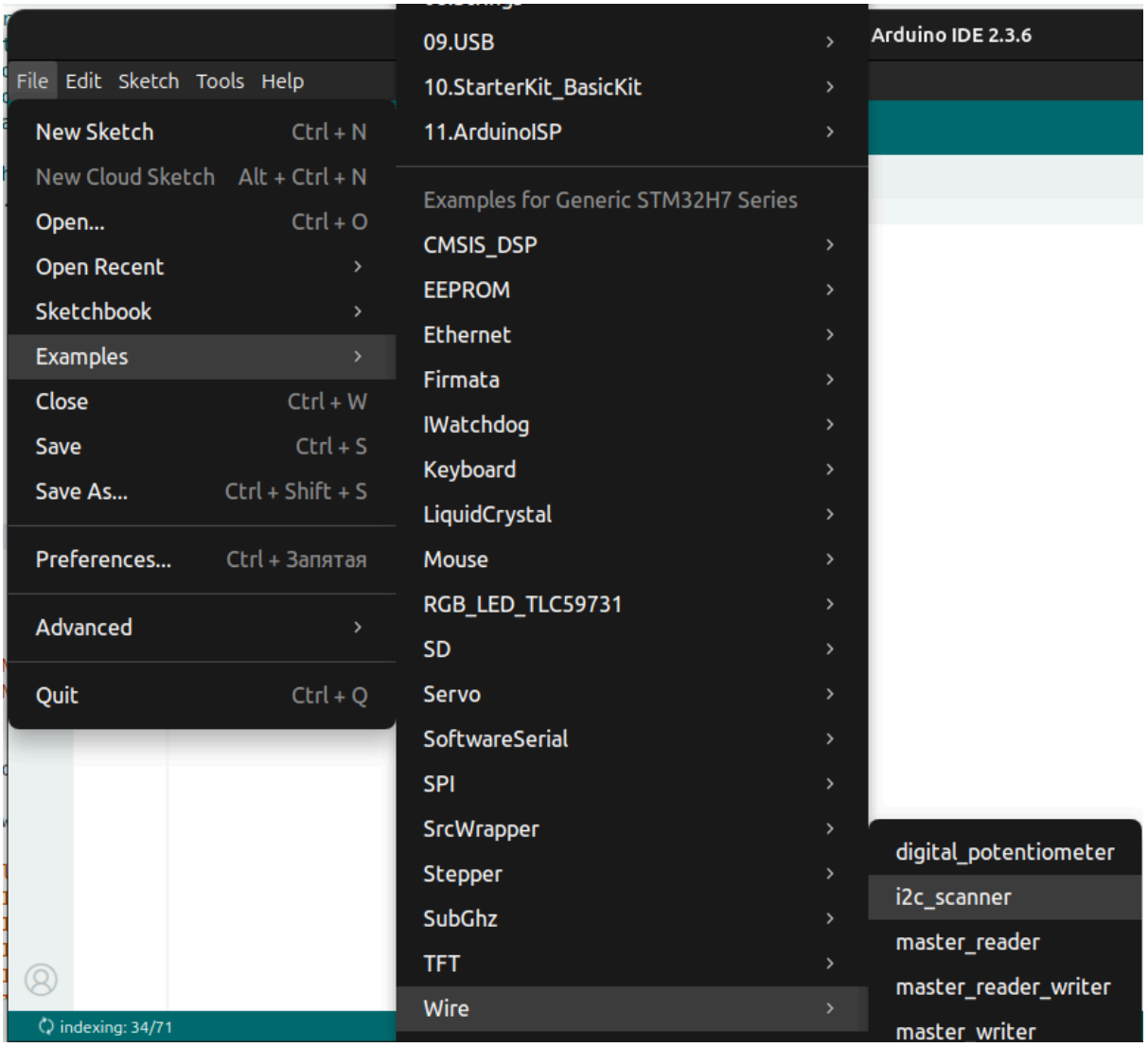
It is recommended to work with sensors using the IntroStratLib library it is described here how to install it).

I2C Scanner

I2C Scanner is an example from the Arduino Wire.h standard library that allows you to set the addresses of devices that are properly connected to the I2C bus. The microcontroller iterates through all 127 seven-bit addresses and tries to establish communication with the device presumably located at each address. If the connection is established successfully, an inscription is displayed in the port monitor that a device with this address is connected to the bus. This sketch is useful to use in some cases:

- when reviewing the FCM board, to see which addresses the service sensors
- are located to check the correct use of the I2C bus in the code, to make sure that the interface object is created correctly and the correct pins are used.
- If incorrect data is received from the sensor, for initial debugging, you can check whether it responds to requests from the host device.

The I2C Scanner sketch can be found in the library examples as follows: in the Arduino IDE, click *File → Examples → Wire → i2c_scanner*.



To work with the FCM board, you need to make some changes to the sketch:

- create an I2C interface object that you want to scan
- create a UART interface object that you want to output data to in order to track the result in the port monitor
- replace the Serial and Wire objects used in the sketch with those you created

If you scan the I2C service bus, then as a result you should see at least four addresses of the service sensors of the FCM board.

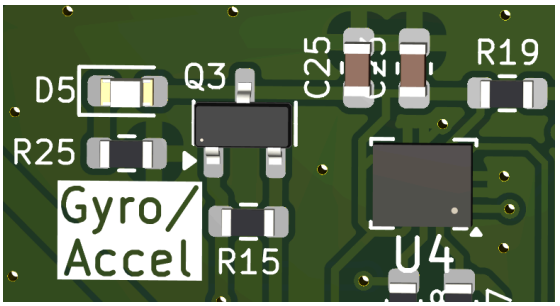
```
->
-> Device found at 0x1C
->
-> Device found at 0x4A
->
-> Device found at 0x6A
->
-> Device found at 0x77
->
```

An example of an I2C1 service bus scan

Gyroscope

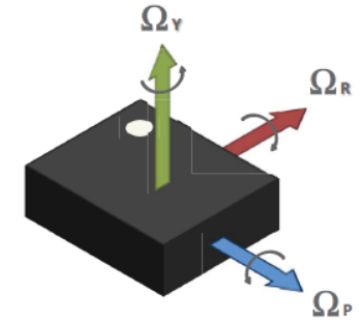
The LSM6DS3 sensor allows you to measure angular velocities along three axes in several ranges with varying accuracy (the smaller the range, the greater the accuracy, since one measured value always occupies 2 bytes in the corresponding registers of the sensor).

The LSM6DS3 sensor is marked on the FCM board as U4. It is located as follows:



U4 – LSM6DS3

According to the sensor specification, the axes around which the angular velocities are measured are arranged as follows:



Direction of detectable angular rate (top view)

Using the key (point) on the sensor housing, you can determine the directions of the measurement axes on the FCM board.

Detailed information about the sensor can be obtained at the link:

https://www.st.com/resource/en/application_note/an5130-lsm6ds3trc-alwayson-3d-accelerometer-and-3d-gyroscope-stmicroelectronics.pdf

To get data from the gyroscope in the sketch, first of all you need to connect the header file:

```
#include <LSM6DS3.h>
```

Then you need to enable the IntroStratLib library namespace:

```
using namespace IntroStratLib;
```

Create an object of the LSM6DS3 class to work with the sensor:

```
LSM6DS3 sm(MYWIRE1, 0x6A);
```

`MYWIRE1` – The I2C service bus object, `0x6A` – the sensor address. The name `sm` can be anything.

In the `setup()` function initialize the I2C service bus and the sensor. The initialization of the sensor in this case sets the settings of the gyroscope. It is recommended to add a small delay after each initialization, for example, 100 ms:

```
MYWIRE1.begin();
delay(100);
sm.InitGyro();
delay(100);
```

Initialization takes place with the default settings:

- Data rate 104 Hz
- Measuring range ±500 dps

You can initialize the sensor with the desired settings by specifying them as method arguments `InitGyro()`. Detailed information about the settings can be found in the library source code `LSM6DS3`.

To get the “raw” angular velocity data (the angular velocity value on one axis takes two bytes), there are RawGX, RawGY and RawGZ methods that return the value of the data type *int16_t*:

```
sm.RawGX();
sm.RawGY();
sm.RawGZ();
```

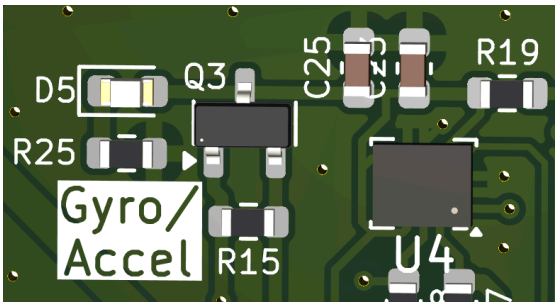
To obtain angular velocity readings in degrees per second, there are GX, GY, and GZ methods that return the value of the data type *float*:

```
sm.GX();
sm.GY();
sm.GZ();
```

Accelerometer

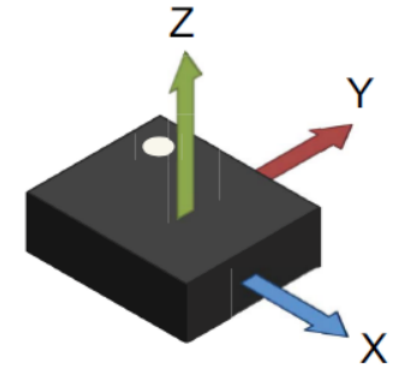
The LSM6DS3 sensor also allows you to measure linear accelerations along three axes in several ranges with varying accuracy (the smaller the range, the greater the accuracy, since one measured value always occupies 2 bytes in the corresponding registers of the sensor).

The LSM6DS3 sensor is marked on the FCM board as U4. It is located as follows:



U4 – LSM6DS3

According to the sensor specification, the axes in the direction of which linear accelerations are measured are arranged as follows:



Direction of detectable acceleration (top view)

Using the key (point) on the sensor housing, you can determine the directions of the measurement axes on the FCM board.

Detailed information about the sensor can be obtained at the link:

https://www.st.com/resource/en/application_note/an5130-lsm6ds3trc-alwayson-3d-accelerometer-and-3d-gyroscope-stmicroelectronics.pdf

To get data from the accelerometer in the sketch, first of all you need to connect the header file:

```
#include <LSM6DS3.h>
```

Then you need to enable the IntroStratLib library namespace:

```
using namespace IntroStratLib;
```

Create an object of the LSM6DS3 class to work with the sensor:

```
LSM6DS3 sm(MYWIRE1, 0x6A);
```

`MYWIRE1` – The I2C service bus object, `0x6A` – the sensor address. The name `sm` can be anything.

In the `setup()` function initialize the I2C service bus and the sensor. The initialization of the sensor in this case sets the settings of the accelerometer. It is recommended to add a small delay after each initialization, for example, 100 ms:

```
MYWIRE1.begin();
delay(100);
sm.InitAccel();
delay(100);
```

Initialization takes place with the default settings:

- Data rate 104 Hz
- Measuring range ±4g

You can initialize the sensor with the desired settings by specifying them as method arguments `InitAccel()`. Detailed information about the settings can be found in the library source code `LSM6DS3`.

To get the “raw” linear acceleration data (the linear acceleration value on one axis takes two bytes), there are RawAX, RawAY and RawAZ methods that return the value of the data type *int16_t*:

```
sm.RawAX();
sm.RawAY();
sm.RawAZ();
```

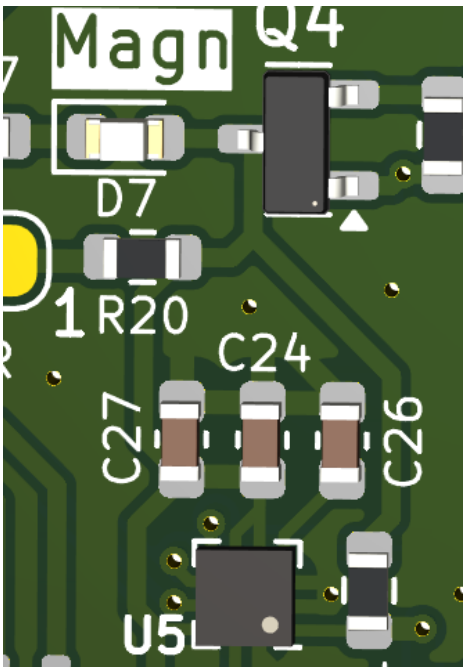
To obtain readings of linear acceleration in fractions of the acceleration of gravity g, there are methods AX, AY and AZ that return a value of the data type *float*:

```
sm.AX();
sm.AY();
sm.AZ();
```

Magnetometer

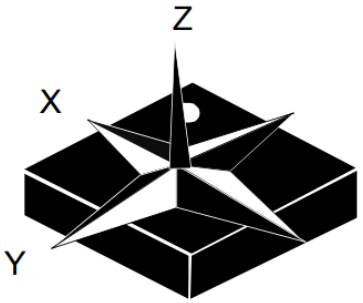
The LIS3MDL sensor allows you to measure magnetic induction along three axes in several ranges with varying accuracy (the smaller the range, the greater the accuracy, since one measured value always occupies 2 bytes in the corresponding registers of the sensor).

The LIS3MDL sensor is marked on the FCM board as U5. It is located as follows:



U5 – LIS3MDL

According to the sensor specification, the axes in the direction of which the magnetic induction is measured are arranged as follows:



Using the key (point) on the sensor housing, you can determine the directions of the measurement axes on the FCM board.

Detailed information about the sensor can be obtained at the link:

https://www.st.com/resource/en/datasheet/lis3mdl.pdf

To receive data from the magnetometer in the sketch, first of all you need to connect the header file:

```
#include <LIS3MDL.h>
```

Then you need to enable the IntroStratLib library namespace:

```
using namespace IntroStratLib;
```

Create an object of the LIS3MDL class to work with the sensor:

```
LIS3MDL magn(MYWIRE1, 0x1C);
```

`MYWIRE1` – the I2C service bus object, `0x1C`– the sensor address. The name `magn` may be anything.

In the `setup()` function initialize the I2C service bus and the sensor. It is recommended to add a small delay after each initialization, for example, 100 ms:

```
MYWIRE1.begin();
delay(100);
magn.Init();
delay(100);
```

Initialization takes place with the default settings:

- Measuring range ±16 Гаусс

You can initialize a sensor with the desired measurement range by specifying it as a method argument `Init()`. Detailed information about the settings can be found in the library source code `LIS3MDL`.

To obtain “raw” magnetic induction data (the value of magnetic induction on one axis takes two bytes), there are RawMX, RawMY and RawMZ methods that return the value of the data type *int16_t*:

```
magn.RawMX();
magn.RawMY();
magn.RawMZ();
```

To obtain magnetic induction readings in Gauss, there are MX, MY, and MZ methods that return a data type *float*:

```
magn.MX();
magn.MY();
magn.MZ();
```

The barometer

The MS5611 sensor allows you to measure temperature and pressure in the range from 10 to 1200 mbar, so that you can calculate altitude even when launching into the stratosphere, in conditions of a very thin atmosphere.

Detailed information about the sensor can be obtained at the link:

https://www.te.com/commerce/DocumentDelivery/DDEController?Action=showdoc&DocId=Data+Sheet%7FMS5611-01BA03%7FB3%7Fpdf%7FEnglish%7FENG_DS_MS5611-01BA03_B3.pdf%7FCAT-BLPS0036

To get data from the barometer in the sketch, first of all you need to connect the header file:

```
#include <MS5611.h>
```

Then you need to enable the IntroStratLib library namespace:

```
using namespace IntroStratLib;
```

Create an object of the `MS5611` class to work with the sensor:

```
MS5611 bar(MYWIRE1, 0x77);
```

`MYWIRE1` – the I2C service bus object, `0x77`– sensor address. The `bar` name can be anything.

In the `setup()` function initialize the I2C service bus and the sensor. It is recommended to add a small delay after each initialization, for example, 100 ms:

```
MYWIRE1.begin();

delay(100);
bar.Init();
delay(100);
```

Initialization takes place with the default settings:

- The oversampling rate (OSR) is 1024, which corresponds to sensor measurement errors of 0.027 mbar and 0.005 °C.

You can initialize the sensor with a different desired measurement accuracy by specifying a different OSR value as the `Init()` method argument. Detailed

information about the settings can be found in the source code of the `MS5611` library.

To get the temperature value in degrees Celsius, the `GetTemperature()` method is used, which returns a value of the float data type:

```
bar.GetTemperature();
```

To get the pressure value in millibars, the `GetPressure()` method is used, which returns a value of the float data type:

```
bar.GetPressure();
```

You can estimate the approximate height value in meters using the `GetHeight()` method, which returns a float data type value:

```
bar.GetHeight();
```

There are various formulas that allow you to calculate altitude using pressure and temperature, and you can calculate them yourself using any of them. The greatest accuracy can be achieved if, before measuring the changing altitude, the initial conditions are taken into account, such as height above ground level, atmospheric pressure, and air temperature at the Earth's surface. Detailed information about the exact calculations can be found at the link:

https://www.amsys-sensor.com/downloads/notes/ms5611-precise-altitude-measurement-with-a-pressure-sensor-module-amsys-509e.pdf

Temperature sensor

The LM75AD sensor allows temperature measurement in the range from -55 °C to +125 °C with a resolution of 0.125 °C. This sensor has a configurable address, which allows you to connect multiple sensors to a single I2C bus. On the platform, such sensors are installed on the flight controller board (address 0x4A) and on the communication board (address 0x4C).

Detailed information about the sensor can be obtained at the link:

https://www.nxp.com/docs/en/data-sheet/LM75A.pdf

To receive data from the temperature sensor in the sketch, first of all you need to connect the header file:

```
#include <LM75A.h>
```

Then you need to enable the IntroStratLib library namespace:

```
using namespace IntroStratLib;
```

Create an object of the `LM75A` class to work with the sensor:

```
LM75A temp(MYWIRE1, 0x4A);
```

`MYWIRE1` – the I2C service bus object, `0x4A`– sensor address. The `temp` name can be anything.

In the `setup()` function initialize the I2C service bus and the sensor. It is recommended to add a small delay after each initialization, for example, 100 ms:

```
MYWIRE1.begin();

delay(100);
temp.Init();
delay(100);
```

To get the temperature value in degrees Celsius, the `GetTemperature()` method is used, which returns a value of the *float data type*:

```
temp.GetTemperature();
```

Since the temperature values from the sensor are quite discrete (the measurement resolution is 0.125 °C), if you multiply the temperature value by 8, it can be placed in 2 bytes (an int16_t type variable) without loss of accuracy. This will save space (the size of a *float variable* is 4 bytes), for example, when sending telemetry data.. To get the temperature in this form, you can use the `GetTemperatureTimes8()` method, which returns a signed integer - the temperature value multiplied by 8..

```
temp.GetTemperatureTimes8();
```

Saving data to a microSD card

There is a slot on the motherboard for connecting a microSD card on which you can save data. This can be useful for logging information during experiments, when the satellite layout works independently, so that useful data is not lost due to possible interruptions in communication over the radio channel.

The microSD card is connected to the microcontroller on the FCM board via the SDMMC interface, a special interface for exchanging data with various types of memory cards, specifically SDMMC2.

To write and read information on a microSD card, it is recommended to use the `STM32SD` library, which in turn is based on the `FatFS` library. It is enough to connect the header file of the `STM32SD` library:

```
#include <STM32SD.h>
```

Since the SD card slot allows you to determine whether a card is currently inserted, you need to determine a constant named `SD_DETECT_PIN` and specify the pin of the microcontroller where the card detection pin is connected. In our case, this is pin PB8.

```
#ifndef SD_DETECT_PIN
#define SD_DETECT_PIN PB8
#endif
```

If the constant named `SD_DETECT_PIN` is not defined in the code, then when working with the library, the card detection check simply does not occur.

You also need to create an object of the `File` class of the `STM32SD` library. You will need it for further work with the microSD card:

```
File dataFile;
```

In the `setup()` function it is necessary to reconfigure the pins of the standard SDMMC interface so that they match the pins used on the board:

```
SD.setDx(PC8, PC9, PC10, PC11);
SD.setCMD(PD2);
SD.setCK(PC12);
```

Next, you need to initialize data exchange with the memory card. We'll do this in a loop, checking the condition for the value returned by the `begin()` method.

```
while (!SD.begin());
```

However, if the card is not inserted into the slot, the `begin()` method returns a non-zero value; thus, the code will not be executed further if there is no memory card. After connecting the memory card to the FCM card, the card must be rebooted.

Now you need to open the file and save the value returned by the `open()` method to a previously created `dataFile` object of the `File` class.

```
dataFile = SD.open("datalog.txt", FILE_WRITE);
```

Method arguments:

- `"datalog.txt"` – the file name can be any
- `FILE_WRITE` – attribute to the opening of the file. With this attribute, the file is opened for writing; writing occurs at the end of the file. You can also use the `FILE_READ` attribute to open a read–only file.

You need to check whether the file has been opened successfully. Only in this case can you attempt to write and read to it.:

```
if (!dataFile) {
   // if the file isn't open, pop up an error
}
```

Use this condition for debugging – in the body of the condition, you can specify the output of an error message to the port monitor.

Let's write *String* format data to a successfully opened file using the `println()` method. After that, you need to call the `flush()` method to make sure that the data is written in full.:

```
String dataString = "hello";
if (dataFile) {
  dataFile.println(dataString);
  dataFile.flush();
}
```

In this way, it is also possible to record data in other data formats.

For more information about how to work with files, check out the `STM32SD` library, you can use examples from it.

LSM6DS3 Gyroscope example:

```
#include <LSM6DS3.h>

using namespace IntroStratLib; // Enable IntroStratLib namespace
```

```
HardwareSerial Serial1(PA10, PA9); // Initialize UART1 at pins PA10 PA9 as Serial 1.

TwoWire Wire1(PB7, PB6); // Initialize I2C1 on pins PB7 PB6 as Wire1.

LSM6DS3 sm(Wire1, 0x6A); // Create LSM6DS3(address on Wire1 is 0x6A) class object as sm.

float gx, gy, gz = 0.0f; // variables for gyro data

void setup() {

  Serial1.begin(115200, SERIAL_8E1); // Start Serial1 with 115200 baudrate and 8 bit parity.

  Wire1.begin(); // Start Wire1

  delay(100); // Wait 100ms

  sm.InitGyro(); //  Initialize gyro

  delay(100); // Wait 100ms

}

void loop() {

  gx = sm.GX(); // Read gyro data on axis X into gx

  gy = sm.GY(); // Read gyro data on axis Y into gy

  gz = sm.GZ(); // Read gyro data on axis Z into gz

  Serial1.print(gx); // Print gyro data on axis X into Serial1 (Bluetooth / LoRa buffer)

  Serial1.print(", "); // print delimiter ‘, ’

  Serial1.print(gy); // Print gyro data on axis Y into Serial1 (Bluetooth / LoRa buffer)

  Serial1.print(", "); // print delimiter ‘, ’

  Serial1.println(gz); // Print gyro data on axis Z into Serial1 (Bluetooth / LoRa buffer) and start new line (sends ‘\r\n’ symbols in the end)

  delay(100); // Wait 100ms so as not to send messages very oftenю

}
```